

Software Architecture

Architecture Means – Principles

BSc



Ingo Arnold

Copyright Notice

© Ingo Arnold. All rights reserved.

You may use these materials for your personal internal reference purposes only.

You may not reuse these materials in creating your own educational offerings or in your own educational situations like courses, lectures, or seminars nor may you distribute, transfer, resell or otherwise make them available to any other person or party without the expressed permission of the author.

URLs or other references to Internet Web sites in the training materials are subject to change without notice. Unless otherwise indicated, all companies, organisations, domain names, email addresses, persons, places and events depicted in the Training Materials are for illustrative purposes only and are fictitious. No real connection is intended or inferred.

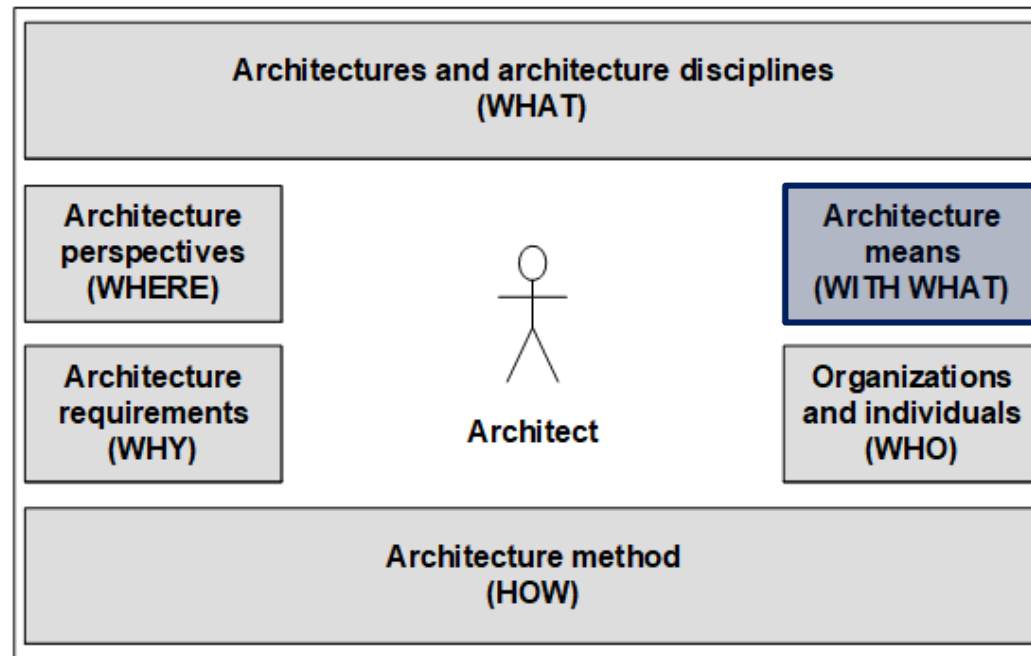
These training materials are provided “as is” and the author makes no warranties, express or implied, with respect to these training materials.

Lecture Opening

Architecture Orientation Framework



Architecture **WITH WHAT** discusses the broad range of architectural means architects use to plan, develop, and operate architectures. Thus what we consider here is the architect's toolbox, if you wish. [Vogel, Arnold et al 2011].



Lecture Opening

Motivation

Software architecture pays special **attention to modifying and extending existing software in order to adapt it to changing needs** (i.e., *maintenance*) or to **use it for similar tasks** (i.e., *reuse*).

Software systems can be divided into encapsulated components which are equipped with interfaces as their explicit abstractions.

These **components can then be replaced individually, combined in different ways, or supplemented.**

In principle, caution is required: extension options that are never needed from the point of view of the business domain are simply superfluous, but cause costs.

Design for reusability only makes sense if there are potential causes for reuse and not organizational, economic or other constraints stand in the way.

Lecture Opening

Motivation

So how can software architecture approaches be combined with real benefits without simultaneously creating too much of a good thing?

Architecture principles guide you in making respective decisions. Some of these principles will be considered in this deck.

Lecture Opening

Learning Objectives

You ...

- can **explain the SOLID architecture principles**.
- can **use the SOLID principles as required to design an architecture**, to justify design decisions, or to evaluate a given design.
- can **question the unreflective application of SOLID principles** and know their limitations.

Lecture Agenda



- Object-Orientation
- Architecture Principles
- Single Responsibility Principle
- Open / Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

Object-Orientation

Overview

Originally, the **object-oriented paradigm has developed from two needs:**

- Programming model based on modelling the real world (low representational gap).
- Objects as cells, from which as by mutation variations can develop (adapt and reuse)

Programming model ...

- The paradigm offers a **programming model that allows the world to be seen as a set of interacting autonomous objects**. These objects encapsulate certain information and respond to (external) events on the basis of this information.
- The **object-oriented model** – with its data encapsulated in objects and operations that can be activated from the outside – **offers the basic building blocks: individual objects with internal states and activatable behavior**.

Objects as a basis for evolutionary fit software systems ...

- At the same time, the increasing size of software systems and respective investments resulted in the need to easily adapt existing programs to new or changing requirements.
- **Software became expensive enough that people wanted to keep it in operation for as long as possible.**
- **By reusing existing software parts, one wanted to reduce the costs** of expensive new software being developed.

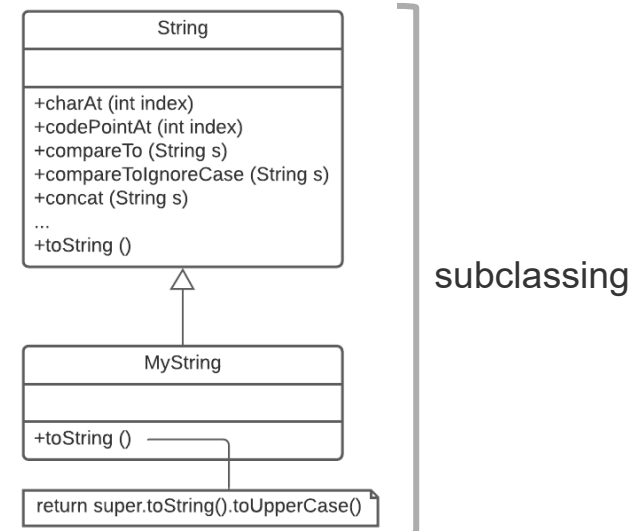
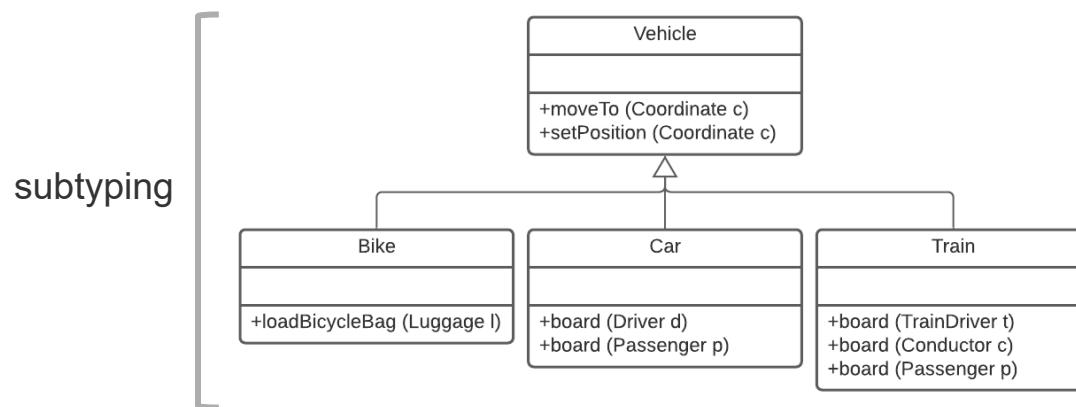
Object-Orientation

Overview

In both cases **the necessity arose to modify the behavior of existing software components while avoiding code changes** or at least limiting them to an utmost extent.

Inheritance **plays a special role for both approaches.**

- In the modelling of *entities* of the real world in the form of *type inheritance* (i.e., *subtyping*), it is used to represent *is-a relations*.
- In the adaptation of existing code in the form of *code inheritance* (i.e., *subclassing*), the behavior of a super object if it is not explicitly overwritten by something new.



Object-Orientation

Overview

Basically, **different types of reuse** can be seen as as a benefit of the object-oriented paradigm:

- **Existing classes are reused** in systems with extended functionality (e.g., JFrame in a window program).
- **Existing classes are used to instantiate additional new objects.**
- **New classes can take over the code of existing classes** (e.g. via inheritance) and (partially) supplement, and/or modify (e.g., String, MyString).

However, **experience shows that intensive *extending and changing* may lead to confusing systems over time.**

As a result **design principles and patterns were shaped.** They are **intended to guide software architects in meaningfully evolving software** and avoid confusing systems over the long term.

Lecture Agenda

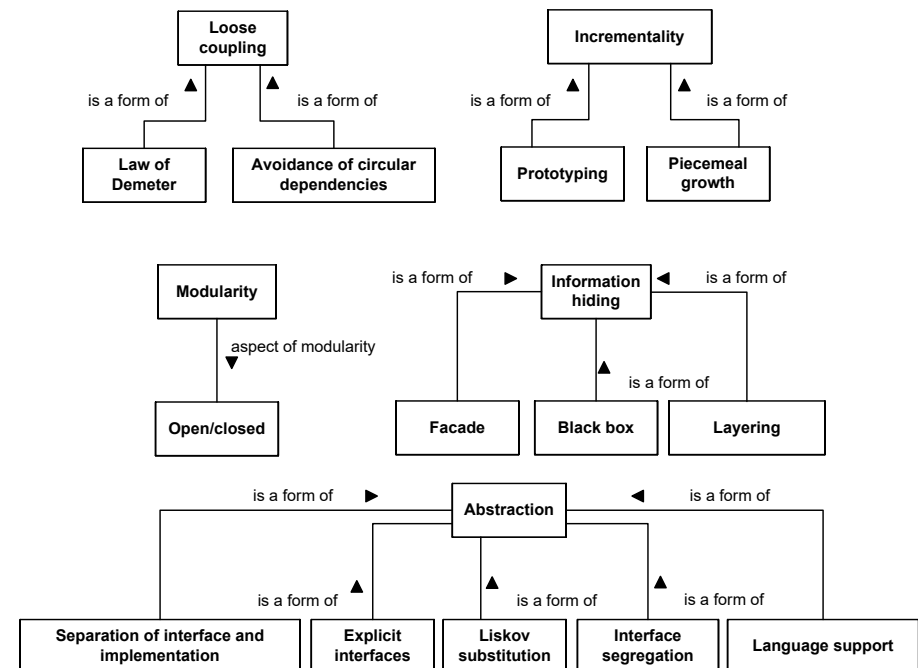
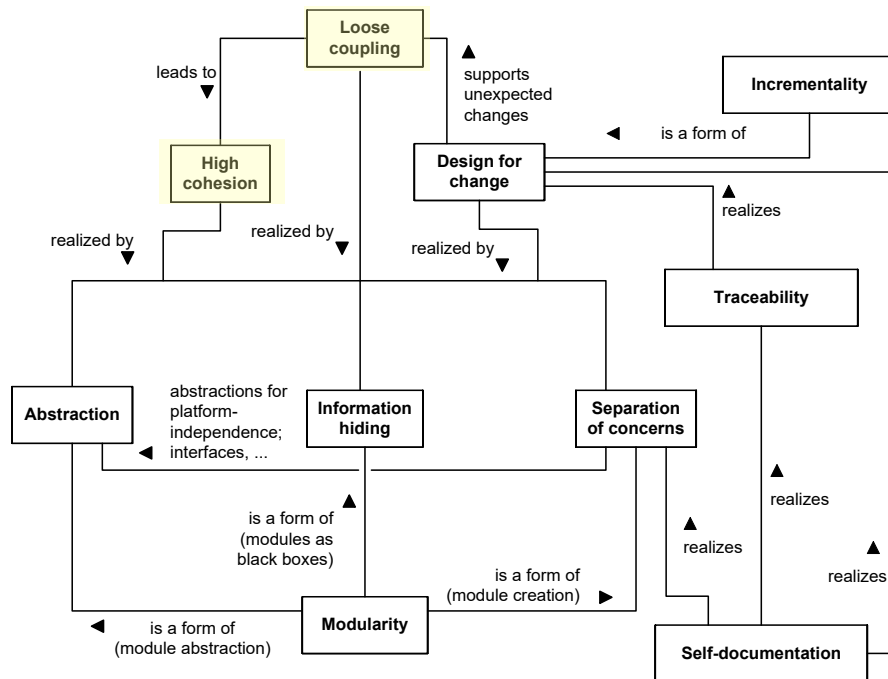


- Object-Orientation
- Architecture Principles
- Single Responsibility Principle
- Open / Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

Architecture Principles Overview

Remember: we have seen in the previous slide deck that architecture principles have many interdependencies and often condition or promote each other.

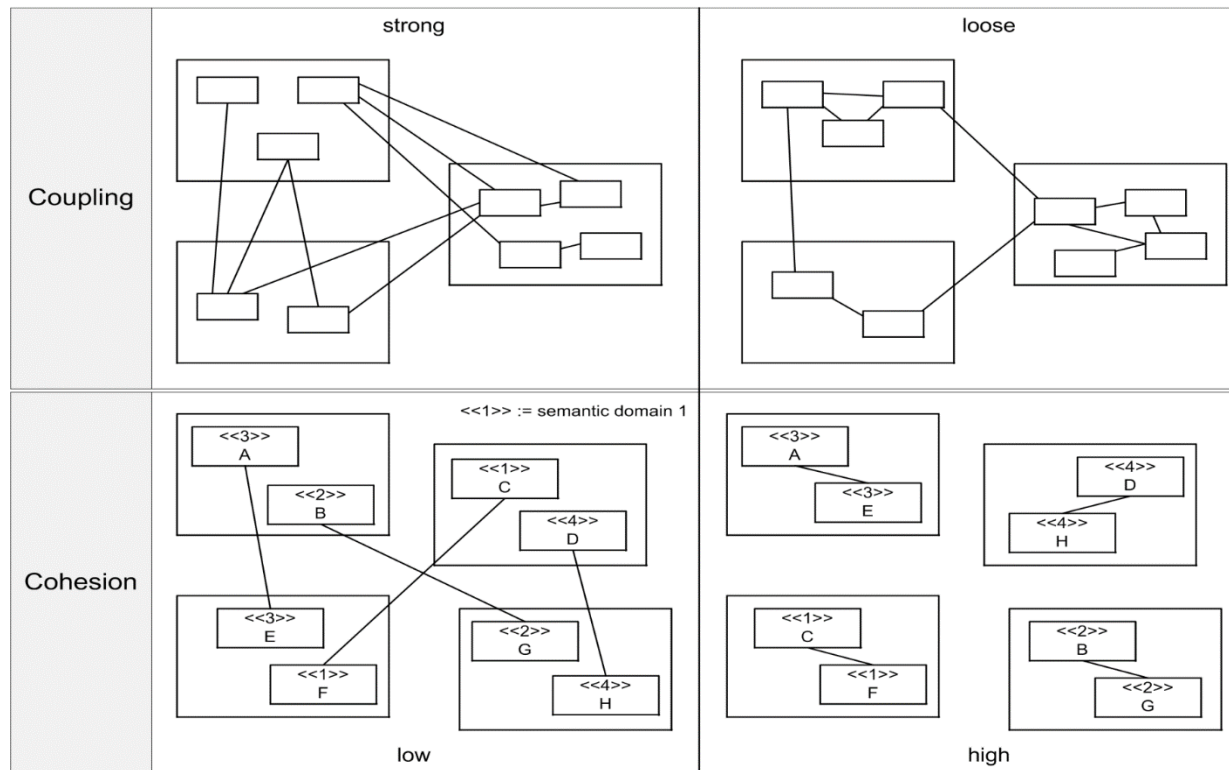
The next slide shows the mutual interdependence of the Loose Coupling and High Cohesion principles.



Architecture Principles

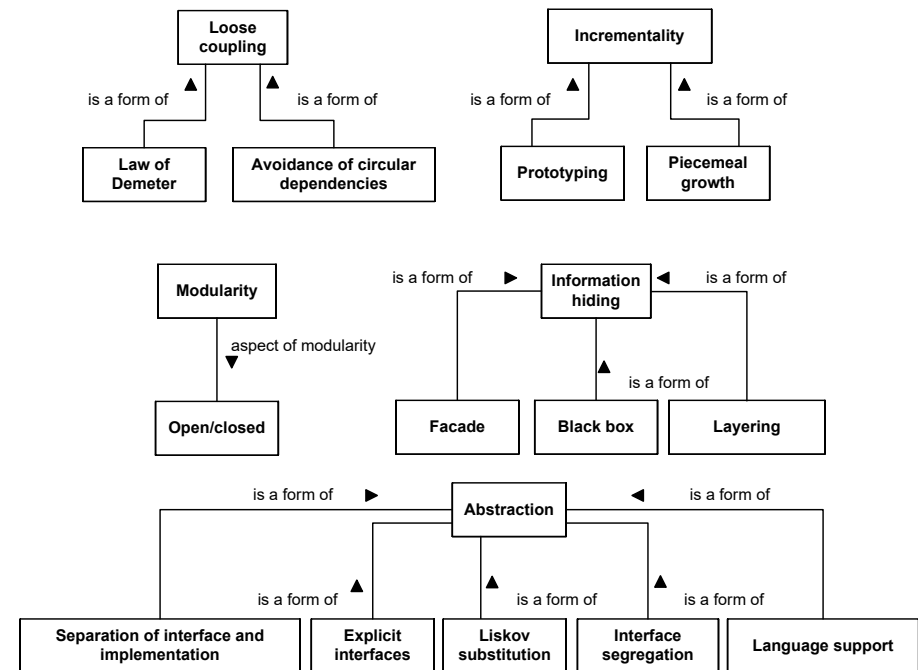
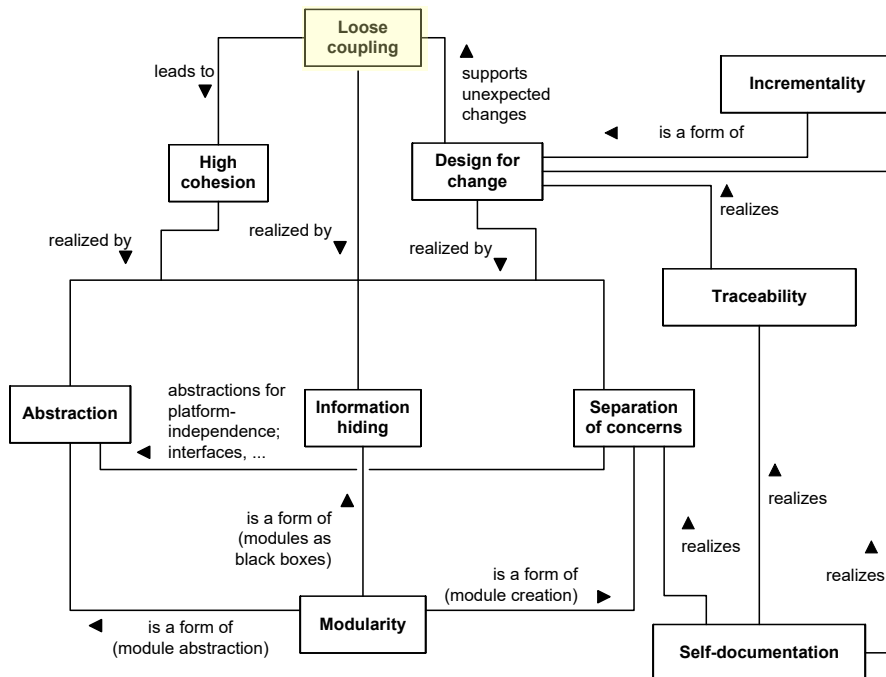
Loose Coupling and High Cohesion

Mutual dependencies exist between many architectural principles, as illustrated here using *coupling* and *cohesion* as examples.



Architecture Principles Overview

The following slides illustrate the many faces that the Loose Coupling can have in practice by a concrete example.





example

Architecture Principles

Loose Coupling

Consider this program and describe what Client binds to in terms of the Database, DatabaseConnection, Query, and ResultSet classes?

```
public class Client {
    public static void main(String[] args) {
        DatabaseConnection dbCon = Database.newConnection("protocol:server:port:db:connection-details");

        dbCon.open();

        // many calls to dbAccess methods (like the one to dbAccessMethod, below) occur throughout the system
        dbAccessMethod(dbCon);

        dbCon.close();
    }
    public static void dbAccessMethod(DatabaseConnection dbCon) {
        Query query = dbCon.createQuery("Select * from Users");
        ResultSet rs = query.perform();

        while(! rs.eof() ) {
            String name = rs.getColumn("name");
            String firstName = rs.getColumn("firstName");
            String city = rs.getColumn("city");
            System.out.println(firstName + " " + name + " " + city);

            rs.next();
        }
    }
}
```



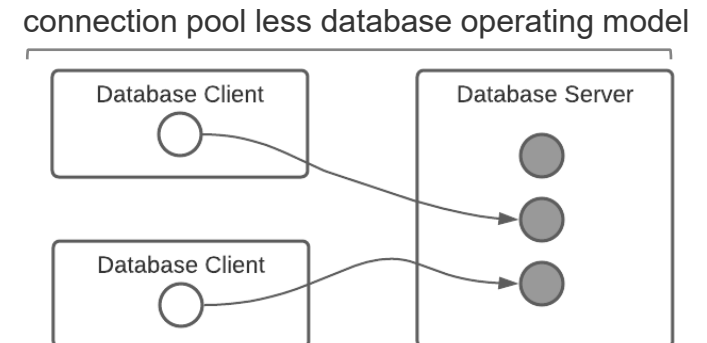
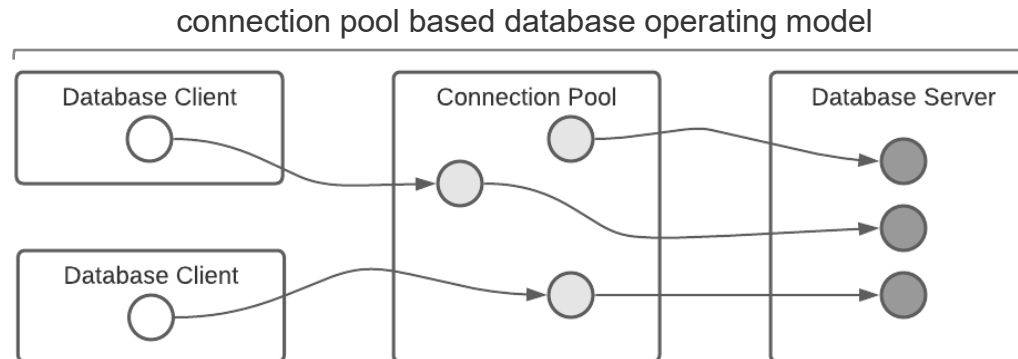
example

Architecture Principles

Loose Coupling

Consider this program and describe what Client binds to in terms of the Database, DatabaseConnection, Query, and ResultSet classes?

- Client may bind to interfaces since DatabaseConnection, Query, ResultSet implementation objects are factorized (e.g., `Database.newConnection()`)
- Client binds to method signature (i.e., method name, parameters order, parameter types, parameter semantics (e.g., *"protocol:server:port:db:connection-details"*), return type, possibly thrown exceptions)
- Client binds to database schema (`dbCon.createQuery("Select * from Users")`, `rs.getColumn("name")`).
- Client binds to method semantics
 - Service semantics (e.g., `dbCon.open()` and `dbCon.close()` in the case of database connection pooling)





example

Architecture Principles

Loose Coupling

- Client binds to method semantics

- Service semantics (e.g., `dbCon.open()` and `dbCon.close()` in the case of database connection pooling)

```
public class Client1 {
    public static void main(String[] args) {
        // many calls to dbAccess methods (like the one to dbAccessMethod, below) occur throughout the
        // system
        dbAccessMethod();
    }
    public static void dbAccessMethod() {
        DatabaseConnection dbCon = Database.newConnection("protocol:server:port:db:connection-details");

        dbCon.open();
        Query query = dbCon.createQuery("Select * from Users");
        ResultSet rs = query.perform();

        while(! rs.eof() ) {
            String name = rs.getColumn("name");
            String firstName = rs.getColumn("firstName");
            String city = rs.getColumn("city");
            System.out.println(firstName + " " + name + " " + city);

            rs.next();
        }
        dbCon.close();
    }
}
```



example

Architecture Principles

Loose Coupling

■ Client binds to method semantics

- Remoting semantics (e.g., `rs.getColumn("name")` versus `rs.getRecord()` in the case of remote objects and the aim to avoid chatty conversations across expensive network and inter-process distances)

```
public class Client2 {
    public static void main(String[] args) {
        dbAccessMethod();
    }
    public static void dbAccessMethod() {
        DatabaseConnection dbCon = Database.newConnection("protocol:server:port:db:connection-details");

        dbCon.open();
        Query query = dbCon.createQuery("Select * from Users");
        ResultSet rs = query.perform();

        while(! rs.eof() ) {
            String record = rs.getRecord();
            System.out.println(record);

            rs.next();
        }
        dbCon.close();
    }
}
```



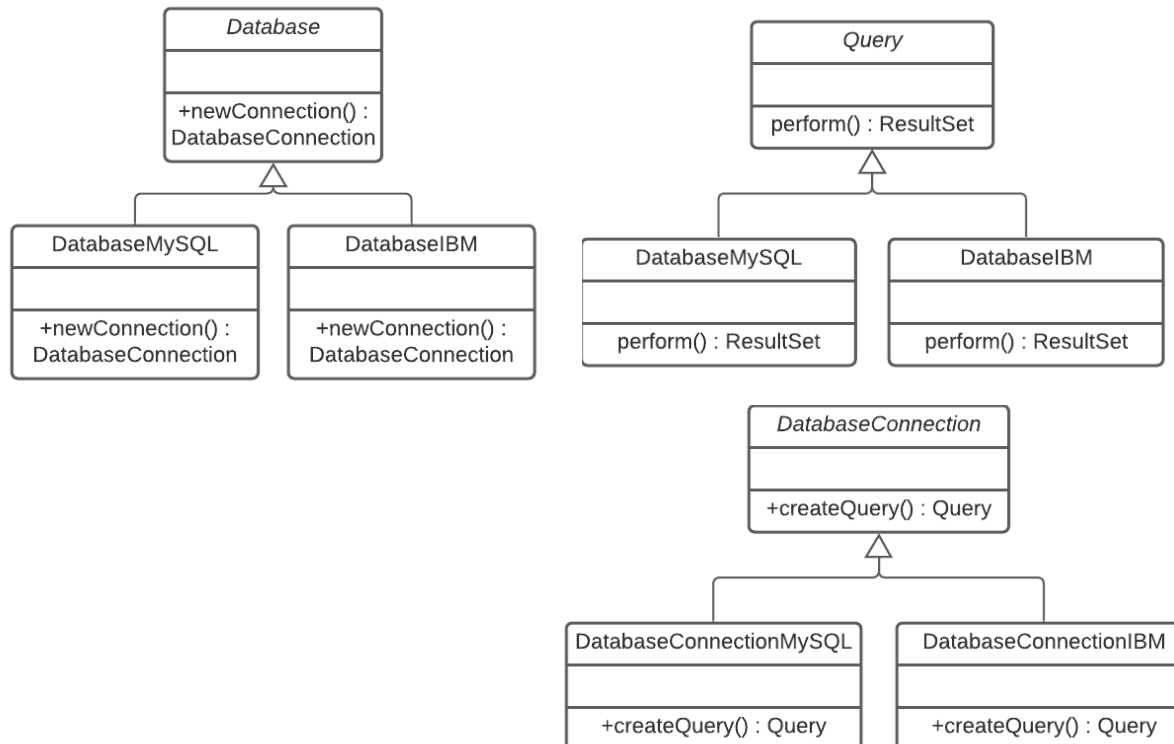
example

Architecture Principles

Loose Coupling

- Client binds to method semantics

- Location semantics (e.g., which implementation is bound to in the factory method `Database.newConnection()` and which implementations are bound to in the subsequent factory methods, like `dbCon.createQuery()` and `query.perform()`)?



Architecture Principles

Overview

The remaining chapters of this deck will focus on a subset of all Architecture Principles known as the SOLID principles ...

- **Single Responsibility Principle.** A class should have only a single responsibility (i.e., only one potential change in the software system specification should be able to affect the specification of the class).
- **Open / Closed Principle.** Software entities should be open for extension, but closed for modification.
- **Liskov Substitution Principle.** Objects in a program should be replaceable with instances of their subtypes without altering the correctness of that program.
- **Interface Segregation Principle.** Many client-specific interfaces are better than one general-purpose interface.
- **Dependency Inversion Principle.** Depend upon abstractions and do not depend upon concretions. Dependency injection is one method of following this principle.

Lecture Agenda



- Object-Orientation
- Architecture Principles
- Single Responsibility Principle
- Open / Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

Single Responsibility Principle

Overview

An early step in object-oriented design is to **figure out what objects should exist**.

In order not to get lost too early in the details and implementation-related aspects such as the exact attributes or signatures of operations, it helps to start with a relatively **abstract assignment of responsibilities in the system**.

The **overall system is divided into individual responsibilities**, each of which is assumed by a dedicated object or class.

Single Responsibility Principle

The Principle

The principle demands that **each class has exactly one responsibility**, i.e. only one type of change in the system specification should be able to affect the class.

One reasoning behind this is that the **code (i.e., the class) of an object that is responsible for several tasks at the same time must be changed every time one of these tasks changes**.

There is a risk that a **change in one area of responsibility will inadvertently affect another area** covered by the same object.

However, **if the principle is adopted and the responsibilities are divided between two objects, the separation is clearer**. If only the code of one of the classes is regenerated (compiled) at all, it is clear that the behavior of the other class is unaffected.

By **access modifiers (i.e., private) one can ensure, that methods, which serve a specific task are not misused** for other tasks. Therefore, a change to such a private method can only affect one task.



example

Single Responsibility Principle

Example

Let us look at a class that represents a simple book entity.

```
public class Book {  
    private String name;  
    private String author;  
    private String text;  
  
    public Book(String name, String author, String text) {  
        this.name = name;  
        this.author = author;  
        this.text = text;  
    }  
    public void replaceWordInText(String regex, String word){  
        text = text.replaceAll(regex, word);  
    }  
    public boolean isWordInText(String word){  
        return text.contains(word);  
    }  
}
```




example

Single Responsibility Principle

Example

Now that we can instantiate a book and edit its text, let us introduce more functionality to also add a print method to our book class.

```
public class Book {  
    private String name;  
    private String author;  
    private String text;  
  
    public Book(String name, String author, String text) {  
        this.name = name;  
        this.author = author;  
        this.text = text;  
    }  
    public void replaceWordInText(String regex, String word){  
        text = text.replaceAll(regex, word);  
    }  
    public boolean isWordInText(String word){  
        return text.contains(word);  
    }  
    public void printTextToConsole() {  
        System.out.println(text);  
    }  
}
```



example

Single Responsibility Principle

Example

How would you resolve the “problem” if you felt that printing text should not be among the responsibilities of the book class?

```
public class Book {  
    private String name;  
    private String author;  
    private String text;  
  
    public Book(String name, String author, String text) {  
        this.name = name;  
        this.author = author;  
        this.text = text;  
    }  
    public void replaceWordInText(String regex, String word){  
        text = text.replaceAll(regex, word);  
    }  
    public boolean isWordInText(String word){  
        return text.contains(word);  
    }  
    public void printTextToConsole() {  
        System.out.println(text);  
    }  
}
```



example

Single Responsibility Principle

Example

If printing text violates the single responsibility principle you can delegate printing to a dedicated class.

```
public class BookPrinter {  
    private Book book = null;  
  
    public BookPrinter(Book book) {  
        this.book = book;  
    }  
    public void printTextToConsole() {  
        System.out.println(book.getText());  
    }  
}
```

```
public class Book {  
    private String name;  
    private String author;  
    private String text;  
    private BookPrinter printer = null;;  
  
    public Book(String name, String author, String text) {  
        this.name = name;  
        this.author = author;  
        this.text = text;  
        this.printer = new BookPrinter(this);  
    }  
    public void replaceWordInText(String regex, String word){  
        text = text.replaceAll(regex, word);  
    }  
    public boolean isWordInText(String word){  
        return text.contains(word);  
    }  
    protected String getText() {  
        return text;  
    }  
    public BookPrinter getBookPrinter() {  
        return printer;  
    }  
}
```

Single Responsibility Principle

Responsibility-driven Design (RDD)

A **constructive design method can be derived from the single responsibility principle**: Starting a list of responsibilities to be assumed by the system, a suitable object is introduced for each of them.

The design is thus driven by the fact that one is looking for responsibility holders. Hence the designation **responsibility-driven design abbreviated with RDD**.

RDD helps to structure beyond the low representational gap (LRG), and if necessary leads to artificial objects, that take on specific responsibilities.

Single Responsibility Principle

Class Responsibility Collaborator (CRC) Cards

The **Class Responsibility Collaborator (CRC)** model [Bellin Suchman 1997] is a collection of standard index cards that have been divided into three sections.

A **class** represents a collection of similar objects, a **responsibility** is something that a class knows or does, and a **collaborator** is another class that a class interacts with to fulfill its responsibilities.

Class Name	
Responsibilities	Collaborators



example

Single Responsibility Principle

Class Responsibility Collaborator (CRC) Cards

Application of CRC technique to a university context

Enrollment	
Mark(s) received Average to date Final grade Student Seminar	Seminar

Transcript	
See the prototype Determine average mark	Student Seminar Professor Enrollment

Student Schedule	
See the prototype	Seminar Professor Student Enrollment Room

Room	
Building Room number Type (Lab, class, ...) Number of Seats Get building name Provide available time slots	Building

Professor	
Name Address Phone number Email address Salary Provide information Seminars instructing	Seminar

Seminar	
Name Seminar number Fees Waiting list Enrolled students Instructor Add student Drop student	Student Professor

Student	
Name Address Phone number Email address Student number Average mark received Validate identifying info Provide list of seminars taken	Enrollment

Building	
Building Name Rooms Provide name Provide list of available rooms for a given time period	Room



example

Single Responsibility Principle

Applying the Principle

Describe the responsibilities entities assume in the gaming platform. Criticize our current approach from an RDD point of view.

- GamePlatform
- UserManagement
- User
- FavouriteList
- GameCatalogue
- Game
- GameResult
- HighScoreList



example

Single Responsibility Principle

Applying the Principle

Describe the responsibilities entities assume in the gaming platform. Criticize our current approach from an RDD point of view.

- GamePlatform
 - The game platform is responsible for providing a façade for user management and the game catalogue.
- UserManagement
 - User Management is responsible for managing users, authenticating users, and displaying all registered users. **Critique:** management and display of users could fall into separate areas of responsibility. Authentication of users as well as any authorization responsibilities could be completely separated from user management.
- User
 - A user represents concrete users of the game platform - including their roles. Furthermore, users have a favourites list. **Critique:** the user role aspect could be completely separated from users.
- FavouriteList
 - A favourites list is responsible for adding games that their user favors. Furthermore, a favourites list is responsible for selecting and displaying games. **Critique:** management, selection as well as display of games in the favourites list could be considered as separate responsibilities.



example

Single Responsibility Principle

Applying the Principle

Describe the responsibilities entities assume in the gaming platform. Criticize our current approach from an RDD point of view.

- GameCatalogue
 - A game catalog is responsible for managing games, selecting a game by a user and displaying all registered games. **Critique:** management, selection as well as display of games could be considered as separate responsibilities.
- Game
 - A game is represented by its name and is responsible for self-registration in a favourites list as well as for being started and played. **Critique:** representation (e.g., by name) and registration of games (e.g., in favourites lists) could be handled in separate responsibilities. Furthermore, game rules could be kept separate from game procedures.



example

Single Responsibility Principle

Applying the Principle

Where do you see the difficulties in applying this principle?



example

Single Responsibility Principle

Applying the Principle

Where do you see the difficulties in applying this principle?

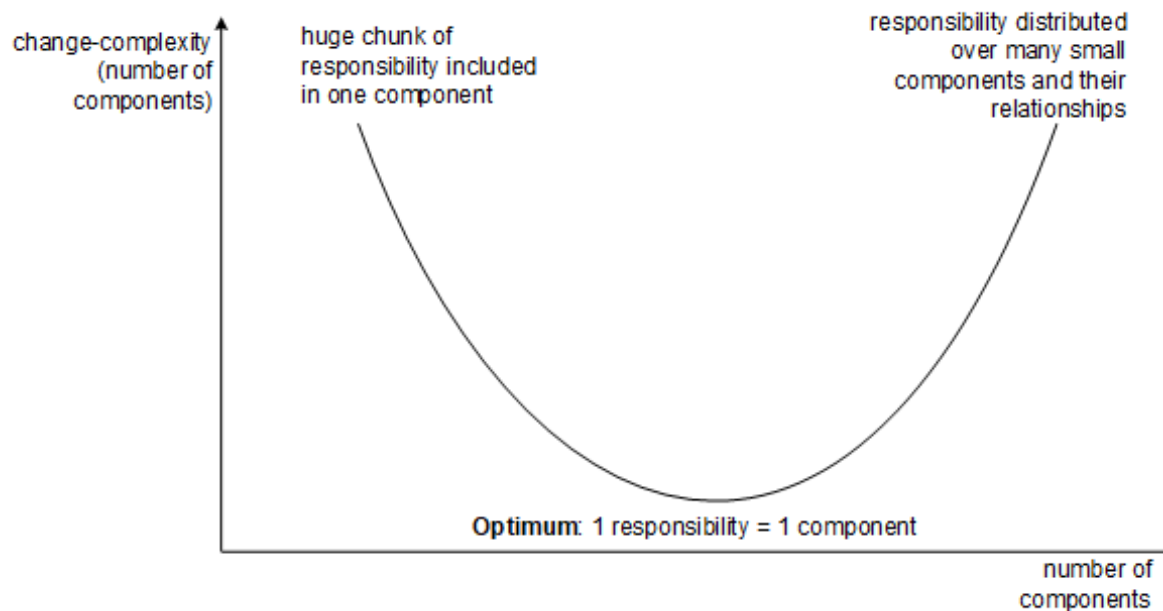
- The problem with applying the single responsibility principle is that **each developer has his own idea of the purpose of a component** under consideration.
- Since we do not have clear instructions on how to implement this principle, **different developers are left with their different interpretations regarding the responsibility of a component.**
- This means that in the end it is always the designer of an application who decides whether something falls within the scope of a component or not.
- When designing a component with the SRP principle in mind, **architects work with their subjective model of the problem domain as well as the business requirements.**

Single Responsibility Principle

Wrap-Up

Typical responsibilities are **creator** (i.e., creating objects), **controller / coordinator** (i.e., coordinating complex processes and activating corresponding operations), and **information expert** (i.e., storing, calculating and providing information).

There is a danger of either combining too many responsibilities in one component or distributing one responsibility too finely across too many components.



Single Responsibility Principle

Wrap-Up

Use the following heuristic to help you decide whether an area of responsibility should be viewed as a single entity or is better broken down.

- What is the likelihood, or frequency, of changes in each area of responsibility?
- Do changes to areas of responsibility, or their components, occur independently of each other?
- Can the division also be made at a finer design level (e.g., within a component)?
- What is the chance that a solution in one area of responsibility will be reused in another context?

Lecture Agenda



- Object-Orientation
- Architecture Principles
- Single Responsibility Principle
- Open / Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

Open / Closed Principle

The Principle

The open / closed principle means **open for extension and closed for modification**.

More specifically this means that **classes should be open to changes in their behavior** and at the same time **closed to changes in the code**.

This principle formulates the desire that **object behavior may change without recompiling their classes**.

Often the open / closed principle leads to case distinctions being replaced by different subtypes, which bundle respective subtype behavior.



example

Open / Closed Principle Example

How do you improve the approach below to avoid distinguishing whether a person is a customer, and in this case storing further attributes?

```
public class Person {  
    private String firstName, lastName;  
  
    public Person(String f, String l) {  
        this.firstName = f;  
        this.lastName = l;  
    }  
    public String toString() {  
        return firstName + " " + lastName;  
    }  
}
```

```
public class CustomerPerson {  
    private String firstName, lastName;  
    private boolean isCustomer;  
    private int turnover = 0;  
  
    public CustomerPerson(String f, String l, boolean isCustomer, int t) {  
        this.firstName = f;  
        this.lastName = l;  
        this.turnover = t;  
        this.isCustomer = isCustomer;  
    }  
    public String toString() {  
        String result = firstName + " " + lastName;  
        if(isCustomer)  
            result += " " + turnover;  
        return result;  
    }  
}
```




example

Open / Closed Principle

Example

For example, instead of distinguishing whether a person is a customer, and in this case storing further attributes, you would subtype the person class.

```
public class Person {  
    private String firstName, lastName;  
  
    public Person(String f, String l) {  
        this.firstName = f;  
        this.lastName = l;  
    }  
    public String toString() {  
        return firstName + " " + lastName;  
    }  
}
```

```
public class Customer extends Person {  
    private int turnover = 0;  
  
    public Customer(String f, String l, int turnover) {  
        super(f,l);  
        this.turnover = turnover;  
    }  
    public String toString() {  
        return super.toString() + " " + turnover;  
    }  
}
```



example

Open / Closed Principle

Example

Another typical approach for the adoption of the open / closed principle is the **employment of interfaces**, in order to **shield component users from component extensions, and/or changes**.

```
public class Car implements Noise {  
    // car-specific attributes  
    // car-specific methods  
    public String makeNoise() {  
        return "Brrrroar";  
    }  
}
```

```
public class Dog implements Noise {  
    // dog-specific attributes  
    // dog-specific methods  
    public String makeNoise() {  
        return "Wuff";  
    }  
}
```

```
public class Cat implements Noise {  
    // cat-specific attributes  
    // cat-specific methods  
    public String makeNoise() {  
        return "Miau";  
    }  
}
```

```
public class Driver {  
    public static void main(String[] args) {  
        List<Noise> noisyList = new ArrayList<> ();  
        noisyList.add(new Car());  
        noisyList.add(new Cat());  
        noisyList.add(new Dog());  
  
        for (int i=0;i<noisyList.size();i++) {  
            System.out.println(noisyList.get(i).makeNoise());  
        }  
    }  
}
```

Open / Closed Principle Design Approach

In the object oriented paradigm, there are basically **two mechanisms for changing the behavior of operations**:

- **Overwriting methods via inheritance** (e.g., method overwriting, template method pattern) or **call redirection via object composition** (e.g., decorator pattern)
- **Outsourcing the behavior into dedicated objects** (e.g., strategy, state, or factory pattern).

An atomic method can only be replaced as a whole.

Alternatively, the algorithmic structure of a method could be represented as a combination of different individual methods, each containing single steps. Here, selected steps can be changed without having to entirely reprogram the overarching method.

However, the method that controls the sequence could also be replaced and the existing individual steps recombined.



example

Open / Closed Principle Design Approach

Overwriting methods via inheritance.

```
public class Base {  
    public void doAll() {  
        step1();  
        step2();  
        step3();  
    }  
    public void step1() { }  
    public void step2() { }  
    public void step3() { }  
}
```

```
public class ExtendedA extends Base {  
    public void step2() {  
        // overwrite behaviour of step2 in base  
    }  
}
```

```
public class ExtendedB extends Base {  
    public void doAll() {  
        step3();  
        step2();  
        step1();  
        stepExtra();  
    }  
    public void stepExtra() {}  
}
```

Open / Closed Principle Design Approach

Problems with inheritance can be reduced if you generally do not overwrite methods with concrete behavior, but at the same time provide possibilities to introduce additional behavior at very specific points in the process.

The exact rule is that non-private, non-static methods of classes that can be subclassed must

- be abstract or
- be final or
- have an empty implementation



example

Open / Closed Principle Design Approach

This is shown in the following example via the **template method pattern**.

```
public abstract class Base {  
    public final void doAll() {  
        step1();  
        step2();  
        step3();  
    }  
    public abstract void step1();  
    public abstract void step2();  
    public abstract void step3();  
}
```

```
public class ExtendedA extends Base {  
    public void step1() { }  
    public void step2() { }  
    public void step3() { }  
}
```

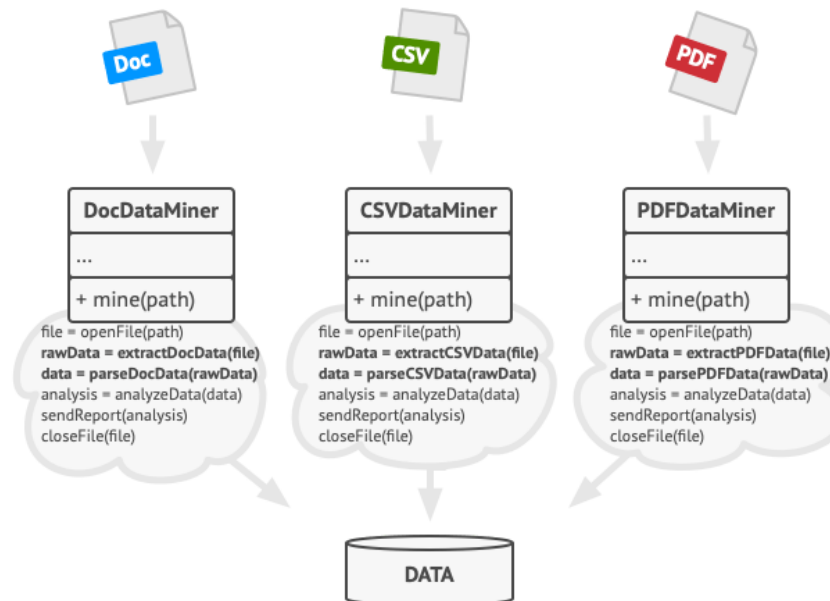


example

Open / Closed Principle Design Approach

How do you cope with the following challenge.

- Imagine that you're creating a data mining application that analyzes corporate documents.
- Users feed the app documents in various formats (PDF, DOC, CSV), and it tries to extract meaningful data from these docs in a uniform format.
- The first version of the app could work only with DOC files. In the following version, it was able to support CSV files. A month later, you “taught” it to extract data from PDF files.

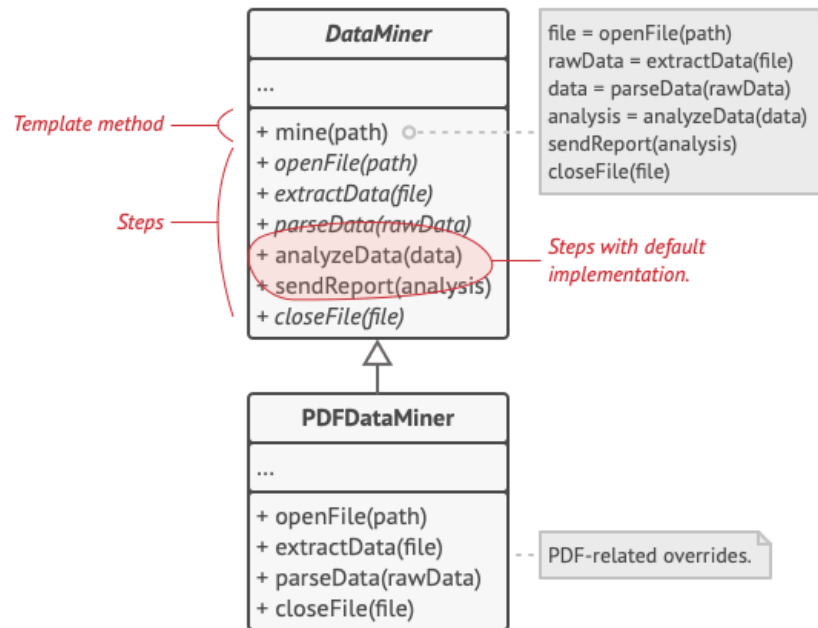




example

Open / Closed Principle Design Approach

How do you cope with the following challenge.





example

Open / Closed Principle Design Approach

How do you cope with the following challenge.

- An HR (human resources) business platform has to provide an employee life-cycle process. The steps of the process include an employment step, which - assuming successful employment - is followed by the steps onboarding the employee, developing the employee, utilizing the employee, evaluating the employee's performance, and finally retiring the employee. The business platform needs to be adaptable to its clients' specific needs (e.g., regarding employment policies).

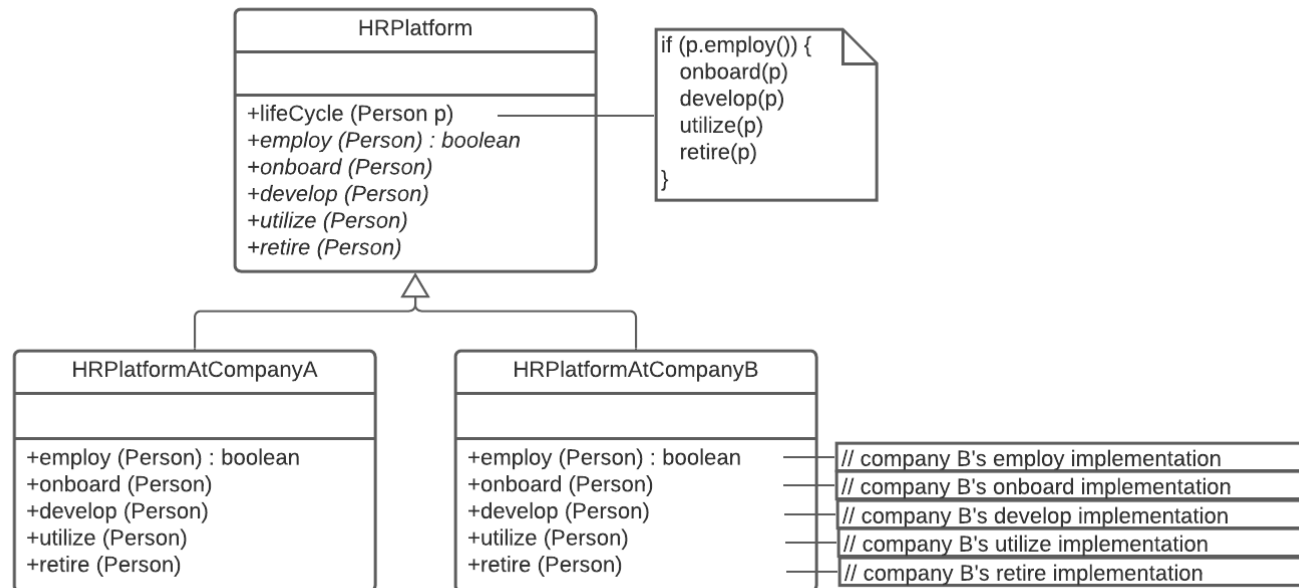


example

Open / Closed Principle Design Approach

How do you cope with the following challenge.

- An HR (human resources) business platform has to provide an employee life-cycle process. The steps of the process include an employment step, which - assuming successful employment - is followed by the steps onboarding the employee, developing the employee, utilizing the employee, evaluating the employee's performance, and finally retiring the employee. The business platform needs to be adaptable to its clients' specific needs (e.g., regarding employment policies).



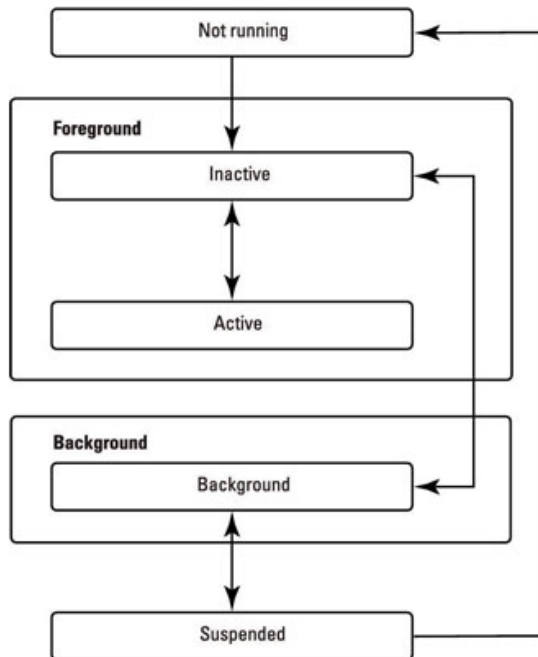


example

Open / Closed Principle Design Approach

How do you cope with the following challenge.

- How could the template method be used to realize the lifecycle of an iOS application?



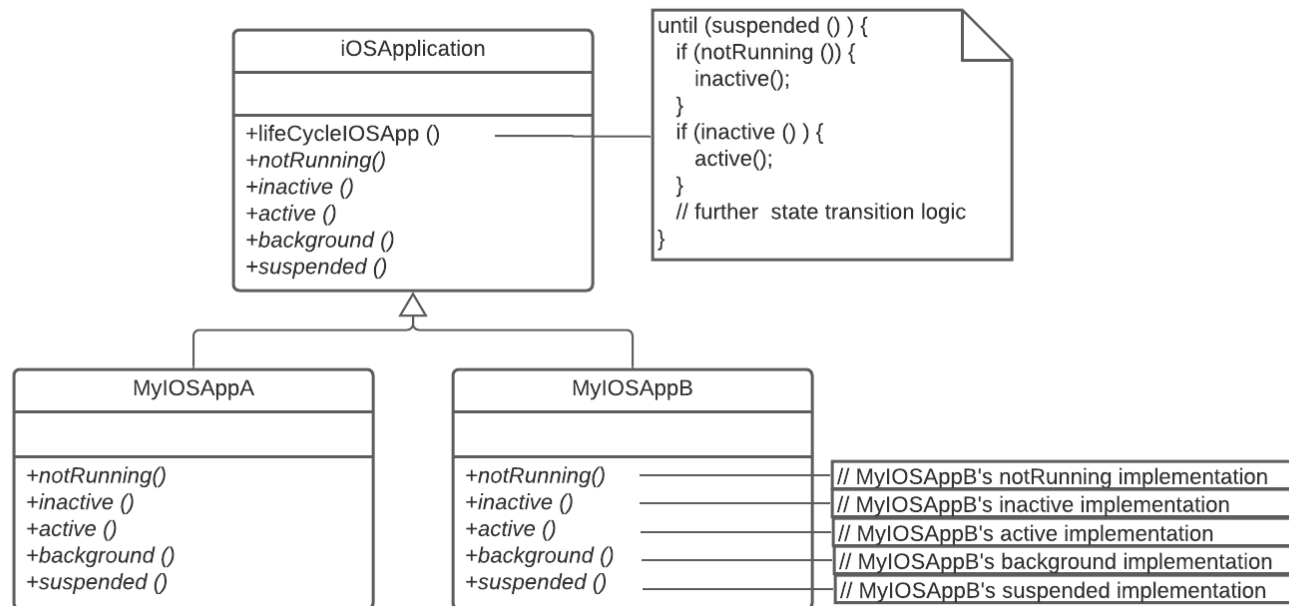


example

Open / Closed Principle Design Approach

How do you cope with the following challenge.

- How could the template method be used to realize the lifecycle of an iOS application?



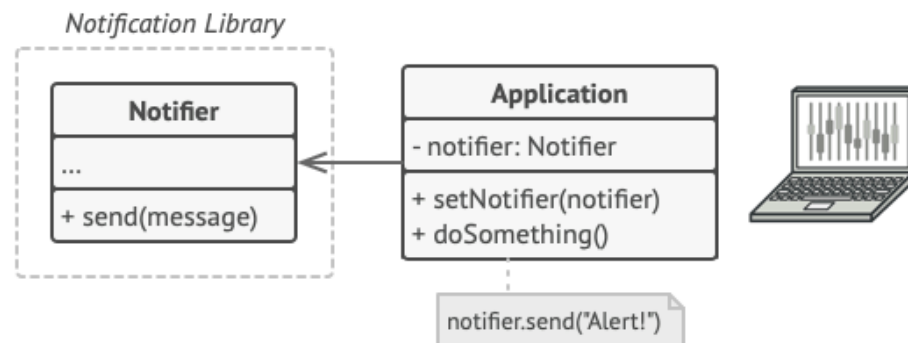


example

Open / Closed Principle Design Approach

Call redirection via object composition based on the **decorator pattern**.

- Imagine that you're working on a notification library which lets other programs notify their users about important events.
- The initial version of the library was based on the Notifier class that had only a few fields, a constructor and a single send method.
- The method could accept a message argument from a client and send the message to a list of emails that were passed to the notifier via its constructor.
- A third-party app which acted as a client was supposed to create and configure the notifier object once, and then use it each time something important happened.



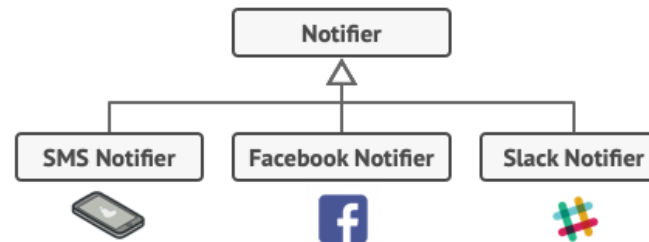


example

Open / Closed Principle Design Approach

Call redirection via object composition based on the **decorator pattern**.

- At some point, you realize that users of the library expect more than just email notifications.
- Many of them would like to receive an SMS about critical issues.
- Others would like to be notified on Facebook and, of course, the corporate users would love to get Slack notifications.
- What you do is extend the Notifier class and put the additional notification methods into new subclasses.
- Now the client was supposed to instantiate the desired notification class and use it for all further notifications.



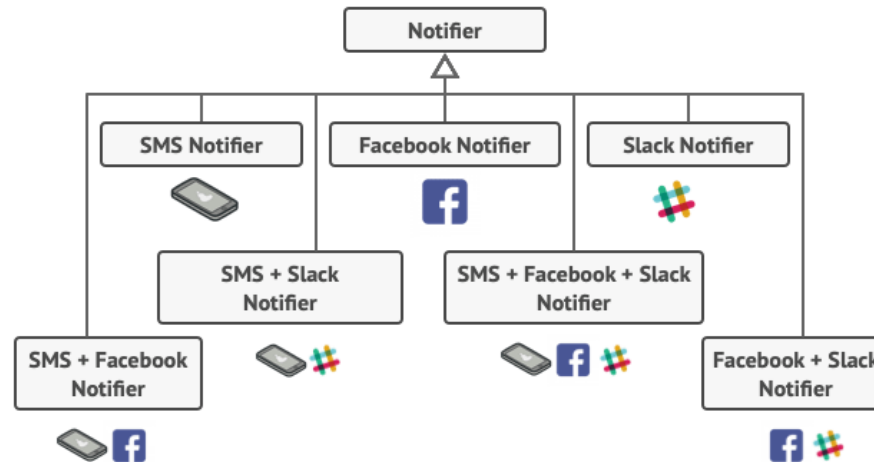


example

Open / Closed Principle Design Approach

Call redirection via object composition based on the **decorator pattern**.

- However, what if someone reasonably asked you: “Why can’t you use several notification types at once? If your house is on fire, you’d probably want to be informed through every channel.”
- You may try to address that problem by creating special subclasses which combined several notification methods within one class.
- However, it becomes apparent that this approach would bloat the code immensely, not only the library code but the client code as well.



Open / Closed Principle Design Approach

Call redirection via object composition based on the **decorator pattern**.

- Extending a class is the first thing that comes to mind when you need to alter an object's behavior. However, inheritance has several serious caveats that you need to be aware of.
 - **Inheritance is static.** You can't alter the behavior of an existing object at runtime. You can only replace the whole object with another one that's created from a different subclass.
 - **Subclasses can have just one parent class.** In most languages, inheritance doesn't let a class inherit behaviours of multiple classes at the same time.
- One of the ways to overcome these caveats is by using Aggregation or Composition instead of Inheritance.
- Both of the alternatives work almost the same way:
 - one object has a reference to another and delegates it some work
 - whereas with inheritance, the object itself is able to do that work, inheriting the behavior from its superclass.
- With this approach you can easily substitute the linked *helper* object with another, changing the behavior of the container at runtime.
- An object can use the behavior of various classes, having references to multiple objects and delegating them all kinds of work.
- Aggregation/composition is the key principle behind many design patterns, including Decorator. On that note, let's return to the pattern discussion.

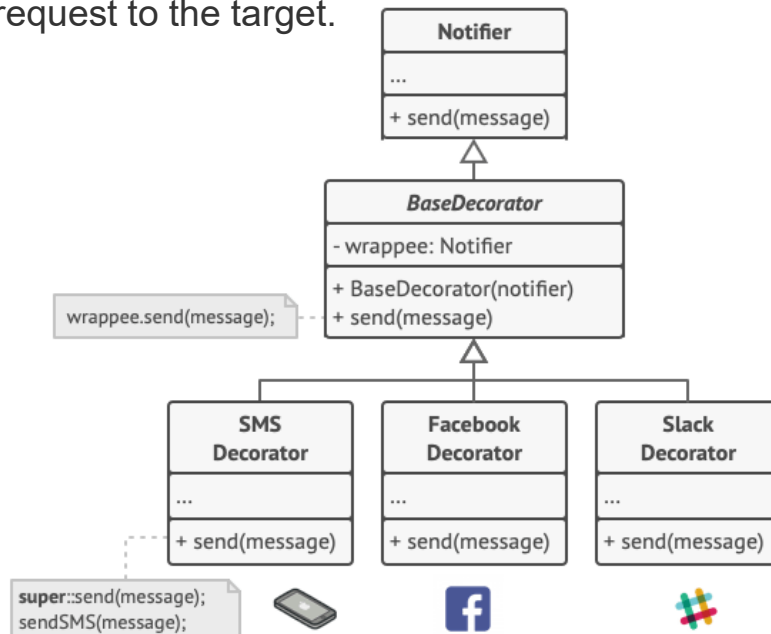


example

Open / Closed Principle Design Approach

Call redirection via object composition based on the **decorator pattern**.

- *Wrapper* is the alternative nickname for the Decorator pattern that clearly expresses the main idea of the pattern.
- A wrapper is an object that can be linked with some target object.
- The wrapper contains the same set of methods as the target and delegates to it all requests it receives.
- However, the wrapper may alter the result by doing something either before or after it passes the request to the target.



```

stack = new Notifier()
if (facebookEnabled)
    stack = new FacebookDecorator(stack)
if (slackEnabled)
    stack = new SlackDecorator(stack)
app.setNotifier(stack)
    
```

```

Application
- notifier: Notifier
+ setNotifier(notifier)
+ doSomething()
    
```

Notifier call: `notifier.send("Alert!")`
// Email → Facebook → Slack

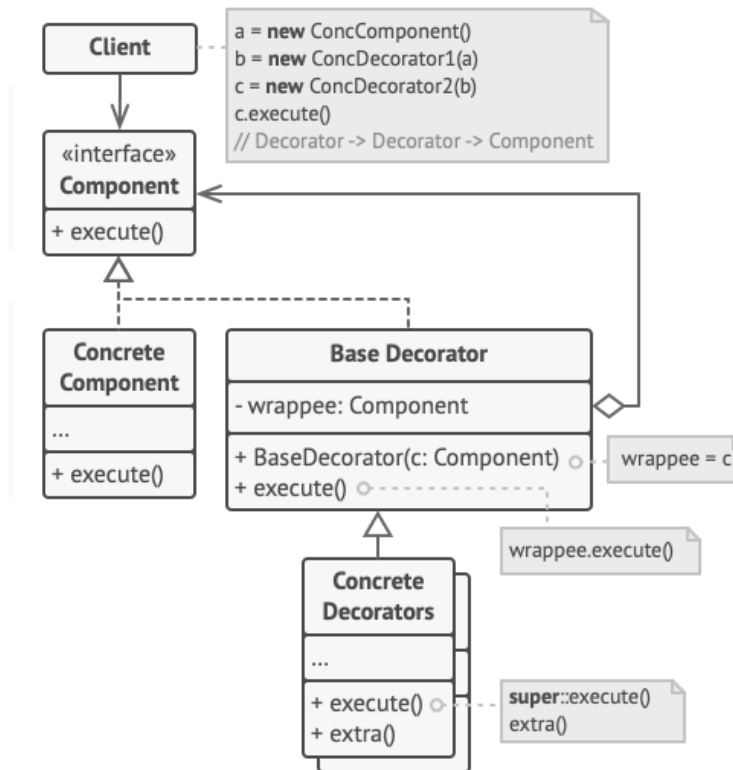




example

Open / Closed Principle Design Approach

Decorator pattern – structure:



Lecture Agenda



- Object-Orientation
- Architecture Principles
- Single Responsibility Principle
- Open / Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

Liskov Substitution Principle

The Principle

While the **open / closed principle** looks at the modifiability of object behavior **from the inside**, the **Liskov principle** views modified objects from the outside.

Regardless of whether a modification is implemented in a subclass or in a decorator, a **changed object is a subtype of the original object**.

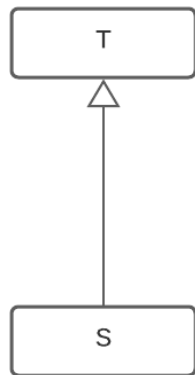
That is, a **changed object can be assigned to variables and method parameters of the supertype**. Users of such variables usually do not know about the existence of the subtype and expect the behavior of the supertype without any restrictions.

Liskov Substitution Principle

The Principle

The principle formulated by Barbara Liskov and Jeanette Wing [Liskov and Wing 1994]

Let $q(x)$ be a property provable about objects x of type T . Then $q(y)$ should be provable for objects y of type S , where S is a subtype of T .



```
public class T {
    public int proof(int i)
    {
        return i;
    }
}
```

```
public class S extends T {
}
```

```
public class Driver {
    public static void main(String[] args) {
        T x = new T();
        S y = new S();

        assert m(x) == m(y);
    }
    public static int m(T s) {
        return s.proof(1);
    }
}
```

Liskov Substitution Principle

The Principle

As a practical heuristic, the Liskov Substitution Principle states the following:

- **Subclasses and derived methods should expect no more and provide no less than their corresponding superclass.** That is, a statement or property of the superclass should not be violated by the subclass.
- **A subclass should be able to take the place of the superclass.** That is, functions with references to base classes must be able to use derived objects without being aware of it.



example

Liskov Substitution Principle

Example

Let us look at an example in Java.

```
public class TransportationVehicle {  
    String name;  
    double speed;  
    Engine engine;  
  
    public String getName() { return name; }  
    public void setName(String name) { this.name = name; }  
    public double getSpeed() { return speed; }  
    public void setSpeed(double speed) { this.speed = speed; }  
    public Engine getEngine() { return engine; }  
    public void setEngine(Engine engine) { this.engine = engine; }  
    public void startEngine() { }  
}
```

```
public class Car extends TransportationVehicle {  
    public void startEngine() {  
        engine.start();  
    }  
}
```

There is no problem here.

A car **is a** transportation vehicle. A car overrides the *startEngine()* method of its superclass.



example

Liskov Substitution Principle

Example

However, what happens if we introduce another transportation vehicle type?

```
public class Bicycle extends TransportationVehicle {  
    public void startEngine() {  
        // a bike (a non-e-Bike) has no engine - problem!  
    }  
}
```

While a bicycle is a transportation vehicle, it does not have an engine and hence, the method *startEngine()* cannot be implemented



example

Liskov Substitution Principle

Example

These are the kinds of **problems** that violation of Liskov Substitution Principle leads to, and they can most usually be recognized by a method that does **nothing**, or even cannot meaningfully be implemented.

The **solution** to these problems is a correct inheritance hierarchy, and in this example we could **solve the problem by differentiating classes of transportation vehicles with and without engines**.

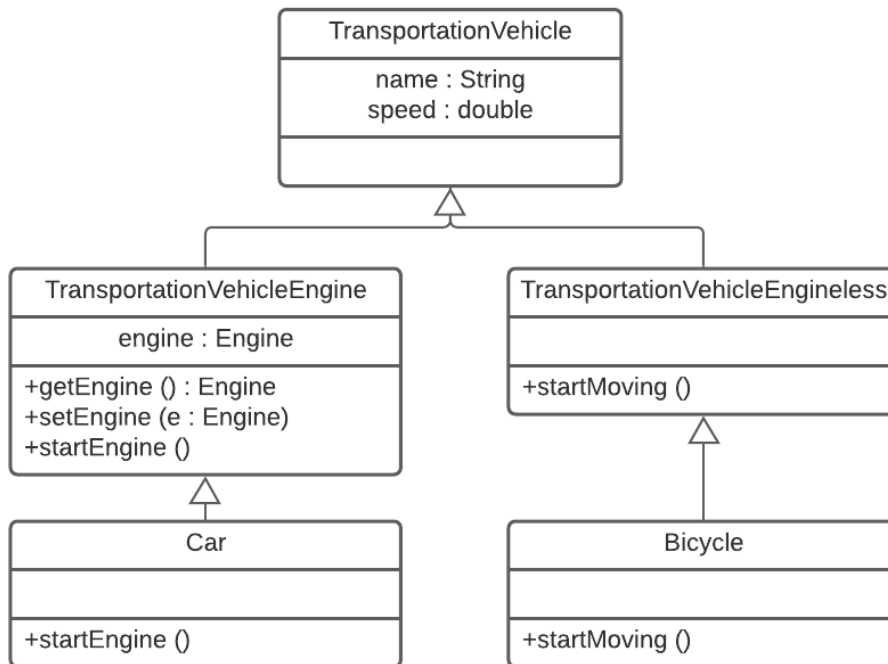
Even though a bicycle is a transportation vehicle, it doesn't have an engine. In this example the mere definition or **abstraction of transportation vehicle is wrong**. Some transportation vehicles should and some should not have an engine.



example

Liskov Substitution Principle Example

Solution visualized:





example

Liskov Substitution Principle

Exercise

Using Liskov's principle, evaluate the following subtyping relation that seems attractive from the code reuse point of view. For example, consider operations that compute or selectively modify the area of such objects.

```
public class Rectangle {  
    private int x, y, width, height;  
  
    public Rectangle(int x, int y, int width, int height) {  
        this.x = x;  
        this.y = y;  
        this.width = width;  
        this.height = height;  
    }  
    public int getX() { return x; }  
    public void setX(int x) { this.x = x; }  
    public int getY() { return y; }  
    public void setY(int y) { this.y = y; }  
    public int getWidth() { return width; }  
    public void setWidth(int width) { this.width = width; }  
    public int getHeight() { return height; }  
    public void setHeight(int height) { this.height = height; }  
}
```

```
public class Square extends Rectangle {  
    public Square(int x, int y, int width) {  
        super(x, y, width, width);  
    }  
  
    public void setWidth(int width) {  
        super.setWidth(width);  
        super.setHeight(width);  
    }  
    public void setHeight(int height) {  
        super.setWidth(height);  
        super.setHeight(height);  
    }  
}
```



example

Liskov Substitution Principle

Exercise

Using Liskov's principle, evaluate the following subtyping relation that seems attractive from the code reuse point of view. For example, consider operations that compute or selectively modify the area of such objects.

```
public class Driver1 {  
    public static void main(String[] args) {  
        Rectangle r = new  
            Rectangle(1,2,50,100);  
        Rectangle s = new Square(1,2,50);  
  
        // works as expected  
        int w = r.getWidth();  
        r.setHeight(2 * w);  
        assert w == r.getWidth();  
  
        // throws exception  
        w = s.getWidth();  
        s.setHeight(2 * w);  
        assert w == s.getWidth();  
    }  
}
```

```
public class Driver2 {  
    public static void main(String[] args) {  
        Rectangle r = new Rectangle(1,2,50,100);  
        Rectangle s = new Square(1,2,50);  
  
        // works as expected  
        int a = getArea(r);  
        doubleArea(r);  
        assert getArea(r) / a == 2;  
  
        // throws exception  
        a = getArea(s);  
        doubleArea(s);  
        assert getArea(s) / a == 2; // 4 is expected instead  
    }  
    public static int getArea(Rectangle r) {  
        return r.getWidth() * r.getHeight();  
    }  
    public static void doubleArea(Rectangle r) {  
        r.setHeight(r.getHeight() * 2);  
    }  
}
```

Lecture Agenda



- Object-Orientation
- Architecture Principles
- Single Responsibility Principle
- Open / Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

Interface Segregation Principle

The Principle

The interface segregation principle (ISP) requires that **common supertypes of objects be defined based on the roles these objects can assume in the system.**

In other words ISP requires that implementing classes are not forced to implement interfaces that they do not really need.

When following the principle, **split interfaces that are very large into smaller and more specific interfaces (i.e., role interfaces)** so that their clients need to know only the methods that are of interest to them.

Each role interface declares one or more methods for a particular behavior (i.e., role). So **instead of implementing one or a few fat interface(s), clients can selectively implement a needed role interface** – an interface whose methods are relevant to them.

Interface Segregation Principle

The Principle

Do not make yourself dependent on aspects you do not need. **Interfaces that contain methods that are not specific to them are called dirty or fat interfaces** and should be avoided.

Many custom interfaces are better than one general-purpose interface. When we have non-coherent interfaces, the principle guides us to create several smaller, coherent interfaces.



example

Interface Segregation Principle

Example

Consider an application that builds different types of toys. Each toy will have a price and colour. Some toys, such as a toy car or toy train can additionally move, while some toys, such as a toy plane can both move and fly.

An interface to define the behaviours of toys is this.

```
public interface Toy {  
    public void setPrice(double price);  
    public void setColor(String color);  
    public void move();  
    public void fly();  
}
```




example

Interface Segregation Principle

Example

A class that represents a toy plane can implement the interface and provide implementations of all the interface methods.

```
public interface Toy {  
    public void setPrice(double price);  
    public void setColor(String color);  
    public void move();  
    public void fly();  
}
```

```
public class ToyPlane implements Toy {  
    public void setPrice(double price) {  
        // implement ToyPlane price setting  
    }  
    public void setColor(String color) {  
        // implement ToyPlane color setting  
    }  
    public void move() {  
        // implement ToyPlane moving on the runway  
    }  
    public void fly() {  
        // implement ToyPlane flying high up in the sky  
    }  
}
```



example

Interface Segregation Principle

Example

However, imagine a class that represents a toy house – this is how such class may look like.

```
public interface Toy {  
    public void setPrice(double price);  
    public void setColor(String color);  
    public void move();  
    public void fly();  
}
```

```
public class ToyHouse implements Toy {  
    public void setPrice(double price) {  
        // implement ToyPlane price setting  
    }  
    public void setColor(String color) {  
        // implement ToyPlane color setting  
    }  
    public void move() { }  
    public void fly() { }  
}
```

ToyHouse needs to provide an implementation of the *move* and *fly* methods, even though it does not require them.

This is a violation of the ISP – such violations affect code readability and confuse designers as well as programmers.



example

Interface Segregation Principle

Example

The solution is to segregate the Toy interface into multiple role interfaces – each for a specific behaviour.

```
public interface Toy {  
    public void setPrice(double price);  
    public void setColor(String color);  
}
```

```
public interface Movable {  
    public void move();  
}
```

```
public interface Flyable {  
    public void fly();  
}
```



example

Interface Segregation Principle

Example

Next we can see how toy houses, cars, and planes are implemented in adherence to the defined interfaces.

```
public class ToyHouse implements Toy {  
    private double price;  
    private String color;  
  
    public void setPrice(double price) {  
        this.price = price;  
    }  
    public void setColor(String color) {  
        this.color = color;  
    }  
    public String toString() {  
        return "ToyHouse - Price: " + price +  
            ", colour: " + color;  
    }  
}
```

```
public class ToyCar implements Toy, Movable {  
    private double price;  
    private String color;  
  
    public void move() {  
        System.out.println("car is moving");  
    }  
    public void setPrice(double price) {  
        this.price = price;  
    }  
    public void setColor(String color) {  
        this.color = color;  
    }  
    public String toString() {  
        return "ToyCar - Price: " + price +  
            ", colour: " + color;  
    }  
}
```



example

Interface Segregation Principle

Example

Next we can see how toy houses, cars, and planes are implemented in adherence to the defined interfaces.

```
public class ToyPlane implements Movable, Toy, Flyable {
    private double price;
    private String color;

    public void fly() {
        System.out.println("plane is flying");
    }
    public void move() {
        System.out.println("plane is moving");
    }
    public void setPrice(double price) {
        this.price = price;
    }
    public void setColor(String color) {
        this.color = color;
    }
    public String toString() {
        return "ToyPlane - Price: " + price +
            ", colour: " + color;
    }
}
```



example

Interface Segregation Principle Example

Putting it all together in the Driver class.

```
public class Driver {  
    public static void main(String[] args) {  
        ToyHouse th = new ToyHouse();  
        ToyCar tc = new ToyCar();  
        ToyPlane tp = new ToyPlane();  
  
        th.setPrice(103.3); th.setColor("red");  
        tc.setPrice(98.5); tc.setColor("blue");  
        tp.setPrice(12.55); tp.setColor("yellow");  
  
        List<Toy> toyList = new ArrayList<Toy>();  
        toyList.add(th); toyList.add(tc); toyList.add(tp);  
  
        for(int i=0;i<toyList.size();i++) {  
            System.out.println(toyList.get(i));  
        }  
  
        List<Flyable> flyableList = new ArrayList<Flyable>();  
        flyableList.add(tp);  
  
        for(int i=0;i<flyableList.size();i++) {  
            System.out.println(flyableList.get(i));  
            flyableList.get(i).fly();  
        }  
    }  
}
```

Interface Segregation Principle

Design Approach

As a design approach for the ISP you ...

- look for the roles which an object assumes in the communication with other objects.
- bundle the operations necessary for these roles in an own interface.

Interfaces thus become role definitions and the *implements declaration* becomes a statement about which roles an object assumes.

Lecture Agenda



- Object-Orientation
- Architecture Principles
- Single Responsibility Principle
- Open / Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

Dependency Inversion Principle

The Principle

In a nutshell the **dependency inversion principle (DIP)** says that you should **define variables and parameters of the weakest possible type** (i.e., such that one does not need type casts for the use of the variables and parameters).

The **DIP relates to the interface segregation principle**: A variable should be typed with exactly the roles that an object assigned to it really needs.

Based on this idea, Robert C. Martin's [Martin et al 2003] **definition of the DIP** consists of two parts:

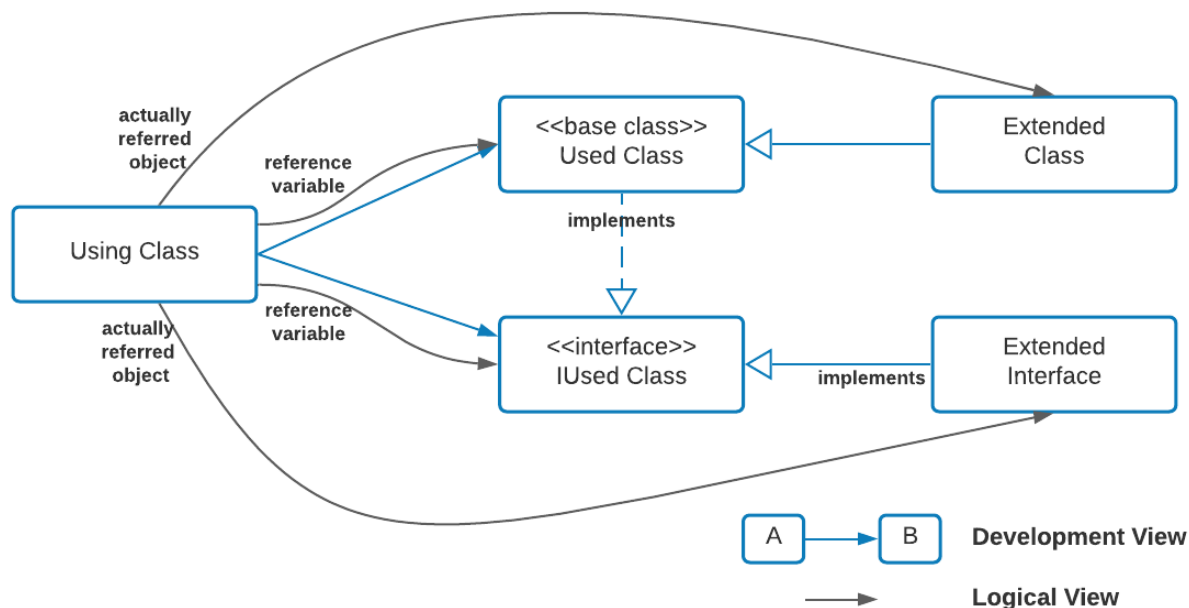
- High-level modules should not depend on low-level modules. Both should depend on abstractions.
- Abstractions should not depend on details. Details should depend on abstractions.

Dependency Inversion Principle

The Principle

The **programmed code is not dependent on the concrete object classes** (i.e., actually referred objects), **but only on role-defining interfaces** (i.e., reference variable).

The same independence applies vice versa for the classes or objects implementing these interfaces (i.e., extended class, extended interface).



Dependency Inversion Principle

The Principle

This basic consideration **also applies to the instantiation of objects**, where the concrete classes to be used must be specified. This is the moment where (at the latest) dependencies arise again.

An **object should not generate needed objects, but receive them "from the outside"**. As long as this is respected, existing objects can work together with new or modified objects without any changes.

Technical tools for this are design patterns such as *factory method*, *abstract factory* or *dependency injection* (e.g., declarative approaches based on XML).

Dependency Inversion Principle

Dependency Injection

Dependency injection (DI) is the concept in which objects get other required objects from the outside.

A class has a dependency on another class, if it uses an instance of this other class (i.e., class dependency). For example, a class which accesses a logger service has a dependency on this service class.

Ideally classes should be as independent as possible from other classes. This increases the possibility of reusing these classes and to be able to test them independently from other classes.

If a class creates an instance of another class via the new operator, it cannot be used (and tested) independently from this other class. This is called a hard dependency.



example

Dependency Inversion Principle

Dependency Injection

The following example shows **a class which has no hard dependencies**, since objects / classes are introduced from the outside and new operator is class-internally used.

```
public class MyClass {  
    private Logger logger;  
  
    public MyClass(Logger logger) {  
        this.logger = logger;  
    }  
    public void log(String logMsg) {  
        logger.info(logMsg);  
    }  
}
```



example

Dependency Inversion Principle

Example

Assume an **example** in which frontend and backend developers are both engaged in projects in order to realize desired solutions.

The frontend developer writes JavaScript and the backend developer writes Java code. Both developers are jointly engaged in a project to help implement the desired solution.

Suggest a design that realizes the outlined situation by introducing BackendDeveloper (method: `writeJavaScript()`), FrontendDeveloper (method: `writeJava()`), and Project (method: `realizeMission()`) classes.



example

Dependency Inversion Principle

Example

An initial design may look like this. However, where does this design violate the DIP?

```
public class FrontEndDeveloper {  
    public void writeJavaScript() {  
        System.out.println("I am a frontend developer and write JavaScript");  
    }  
}
```

```
public class BackEndDeveloper {  
    public void writeJava() {  
        System.out.println("I am a backend developer and write Java");  
    }  
}
```

```
public class Project {  
    private BackEndDeveloper bd = new BackEndDeveloper();  
    private FrontEndDeveloper fd = new FrontEndDeveloper();  
  
    public void realizeMission() {  
        bd.writeJava();  
        fd.writeJavaScript();  
    }  
}
```



example

Dependency Inversion Principle

Example

Where does this design violate the DIP?

```
public class Project {  
    private BackEndDeveloper bd = new BackEndDeveloper();  
    private FrontEndDeveloper fd = new FrontEndDeveloper();  
  
    public void realizeMission() {  
        bd.writeJava();  
        fd.writeJavaScript();  
    }  
}
```

Project is a high-level module and depends on the low-level modules **BackEndDeveloper** and **FrontEndDeveloper**.

This violates the first part of the DIP principle: High-level modules should not depend on low-level modules.



example

Dependency Inversion Principle

Example

Where does this design violate the DIP?

```
public class Project {  
    private BackEndDeveloper bd = new BackEndDeveloper();  
    private FrontEndDeveloper fd = new FrontEndDeveloper();  
  
    public void realizeMission() {  
        bd.writeJava();  
        fd.writeJavaScript();  
    }  
}
```

Further, in the **realizeMission()** method of the project module the methods **writeJava()** and **writeJavaScript()** are bound to their corresponding classes.

Regarding the scope of projects, those **methods are details since they are forms of development.**

This violates the second part of the DIP principle: Abstractions should not depend on details.



example

Dependency Inversion Principle

Example

How does a design look like in which the DIP is not violated?



example

Dependency Inversion Principle

Example

How does a design look like in which the DIP is not violated?

```
public interface Developer {  
    public void develop();  
}
```

First we introduce an abstraction for both types (forms) of developers

```
public class FrontEndDeveloper implements Developer {  
    public void develop() {  
        writeJavaScript();  
    }  
    private void writeJavaScript() {  
        System.out.println("I am a frontend developer and write JavaScript");  
    }  
}
```

Next we refactor FrontEndDeveloper and BackEndDeveloper modules to implement the abstraction (i.e., Developer interface).

```
public class BackEndDeveloper implements Developer {  
    public void develop() {  
        writeJava();  
    }  
    private void writeJava() {  
        System.out.println("I am a backend developer and write Java");  
    }  
}
```



example

Dependency Inversion Principle

Example

How does a design look like in which the DIP is not violated?

```
public class Project {  
    private Developer bd = null;  
    private Developer fd = null;  
  
    public Project(Developer frontEndDeveloper, Developer backEndDeveloper) {  
        bd = backEndDeveloper;  
        fd = frontEndDeveloper;  
    }  
    public void realizeMission() {  
        bd.develop();  
        fd.develop();  
    }  
}
```

Finally we **refactor the Project module** such that it **does not depend on the low-level modules** FrontEndDeveloper and the BackendDeveloper nor on their details (i.e., writeJava() and writeJavaScript()) but instead on the Developer abstraction.



example

Dependency Inversion Principle

Example

If we wanted to generalize projects and be able to engage many different types of developers. If we further introduce a new SQL-Developer who is coding SQL we would extend our improved design.as ...

```
public class SQLDeveloper implements Developer {  
    public void develop() {  
        writeSQL();  
    }  
    public void writeSQL() {  
        System.out.println("I am a database developer and write SQL");  
    }  
}
```

Introducing a **new SQLDeveloper module**.

```
public class Project {  
    private List<Developer> developers = new ArrayList<Developer>();  
  
    public Project(List<Developer> developers) {  
        this.developers = developers;  
    }  
    public void realizeMission() {  
        developers.forEach(d->d.develop());  
    }  
}
```

The project module we would refactor such that two dedicated developer types are represented by a respective list.



example

Dependency Inversion Principle

Example

If we wanted to generalize projects and be able to engage many different types of developers. If we further introduce a new SQL-Developer who is coding SQL we would extend our improved design as ...

```
public class Driver {  
    public static void main(String[] args) {  
        List<Developer> developers = new ArrayList<Developer>();  
  
        developers.add(new FrontEndDeveloper());  
        developers.add(new BackEndDeveloper());  
        developers.add(new SQLDeveloper());  
  
        Project project = new Project(developers);  
        project.realizeMission();  
    }  
}
```

Now we put everything together. In a main method concrete low-level modules are created. Next the high-level module project is equipped with them.

Finally, the Project module does not depend on lower level modules, but rather abstractions. Also, low-level modules and their details depend on abstractions.

Dependency Inversion Principle

Design Approach

In order **to comply with the DIP, you have to create the necessary abstractions in the type hierarchy first** – this means the abstractions you want to use when typing variables and parameters.

Role models you have developed in the context of the interface segregation principle (i.e., program to interfaces) may be helpful.

Abstract types should be formulated as interfaces (i.e., not as abstract classes).

The primary **purpose of abstract classes is to provide code for other classes via inheritance** (i.e., sub-classing). However, this is a task independent of subtyping, which should therefore be separated.

Finally you need to **pay attention to introducing variables and parameters as abstract types wherever possible**.

Wrap-Up

■ Object-Orientation and Motivation

- **Object orientation emerged to model the real world** (low representational gap (LRG)) and to **enable evolutionary reuse of existing software**.
- **Systems** should be **designed** for **adaptability** and **maintainability**, **avoiding unnecessary complexity**.
- **Reuse** and **extensibility** must be **business-driven**, **not pursued as a goal in itself**.
- **Architectural principles** help **balance flexibility** and **simplicity**, **guiding design decisions** that remain sustainable.

Wrap-Up

■ Core Architectural Principles

- **Architectural principles** are **experience-based rules** that **promote maintainable, evolvable, and understandable systems**.
- They are **interrelated** – for example, **loose coupling supports high cohesion** and **modularity**.
- Key Relationships
 - **Modularity**. Enables black-box design and separation of concerns.
 - **Abstraction**. Defines clear boundaries between interface and implementation.
 - **Information Hiding**. Encapsulates internal details, improving change tolerance.
 - **Loose Coupling** ↔ **High Cohesion**. Minimal dependencies, strong internal unity.
 - **Incrementality**. Systems evolve stepwise with minimal rework.
 - **Design for Change**. Anticipate and localize future modifications.

■ Loose Coupling (Detailed Example)

- **Goal: minimize dependencies between system parts**.
- **Dependencies** can **exist** at **several levels**:
 - **Type binding** (interfaces vs. concrete classes).
 - **Location binding Method binding** (signatures, semantics).
 - (runtime dependencies, e.g. remoting).

Wrap-Up

- **Loose Coupling (Detailed Example) – continued**
 - **Excessive coupling reduces flexibility, reusability, and testability.**
 - **Techniques to reduce coupling:**
 - Introduce **interfaces** or **abstract factories**.
 - Use **façades**, **adapters**, and **dependency injection**.
 - **Separate configuration** from **logic** (e.g., avoid hardcoded connections).
- **SOLID Principles**
 - **Single Responsibility Principle (SRP)**
 - **Each class** should have **exactly one reason to change** – **one area of responsibility**.
 - **Reduces side effects** and **clarifies system structure**.
 - **Supports Responsibility-Driven Design (RDD)** (method could be: Class-Responsibility-Collaborator (CRC) cards).
 - **Typical responsibilities:** **creator**, **controller**, **information expert**.
 - **Risk is over-fragmentation; balance cohesion and simplicity.**

Wrap-Up

■ SOLID Principles – continued

• Open / Closed Principle (OCP)

- **Software entities** should be **open** for **extension** but **closed** for **modification**.
- **Behaviour** can **change without altering existing code**.
- **Techniques**
 - **Inheritance** (e.g., template method pattern).
 - **Composition** and **delegation** (e.g., strategy, state, decorator patterns).
- **Encourages polymorphism** and **plug-in architectures**.
- **Avoids case distinctions** and **repeated modifications** of **base classes**.

• Liskov Substitution Principle (LSP)

- **Subtypes must be replaceable** for their **supertypes without changing program correctness**.
- A **subclass must not violate contracts** or **expectations defined** by its **base type**.
- **Violations occur** when **methods cannot be meaningfully implemented** (e.g., `Bicycle.startEngine()`).
- **Fix: design correct hierarchies** and **use interfaces** to **separate incompatible concerns**.

Wrap-Up

■ SOLID Principles – continued

• Interface Segregation Principle (ISP)

- **Clients** should **not depend** on **methods they do not use**.
- **Prefer many small, role-specific interfaces over few large general-purpose interfaces**.
- **Promotes modularity, reusability, and testability**.
- **Each interface defines a specific role; classes may implement multiple roles as needed**.

• Dependency Inversion Principle (DIP)

- **High-level modules** should **not depend** on **low-level modules** – **both** should **depend on abstractions**.
- **Abstractions** should **not depend** on **details**; **details** should **depend on abstractions**.
- **Achieved by**
 - **Interfaces** and **abstractions between layers**.
 - **Dependency Injection (DI)** and **Factory Patterns** to **supply dependencies externally**.
- **Improves testability, flexibility, and isolation of changes**.

Wrap-Up

- **Design Patterns Used in Examples**
 - **Template Method** – defines algorithm skeleton, subclasses override steps.
 - **Decorator / Wrapper** – dynamically extend functionality via composition.
 - **Strategy** – interchangeable algorithms encapsulated in separate classes.
 - **Adapter** – mediates between incompatible interfaces.
 - **Factory / Dependency Injection** – decouple creation from usage.
- **Learning Objectives (Exam Focus) – Students should be able to ...**
 - **Define** and **explain** each **SOLID** principle and its motivation.
 - **Recognize violations** of these principles in example code.
 - **Apply object-oriented design principles** to improve coupling, cohesion, and flexibility.
 - **Compare inheritance** and **composition** as mechanisms for extensibility.
 - **Identify appropriate design patterns** supporting each principle.
 - **Relate** the principles to real-world architectural qualities: maintainability, reusability, testability, and evolvability.

Questions



Bibliography

Lecture

[Bellin and Suchman 1997]

Bellin, David; Simone, Suchman, *The CRC Card Book*, Boston, Addison-Wesley Professional, 1997

[Liskov and Wing 1994]

Liskov, Barbara H.; Wing, Jeannette M., *A behavioural notion of subtyping*, in *ACM Transactions on Programming Languages and Systems (TOPLAS)*, Volume 16 Issue 6, Nov. 1994

[Martin et al 2003]

Martin, Robert C.; Rabaey, Jan M. ; Chandrakasan, Anantha P. ; Nikolić, Borivoje, *Agile Software Development - Principles, Patterns, and Practices*, Amsterdam, Pearson Education, 2003